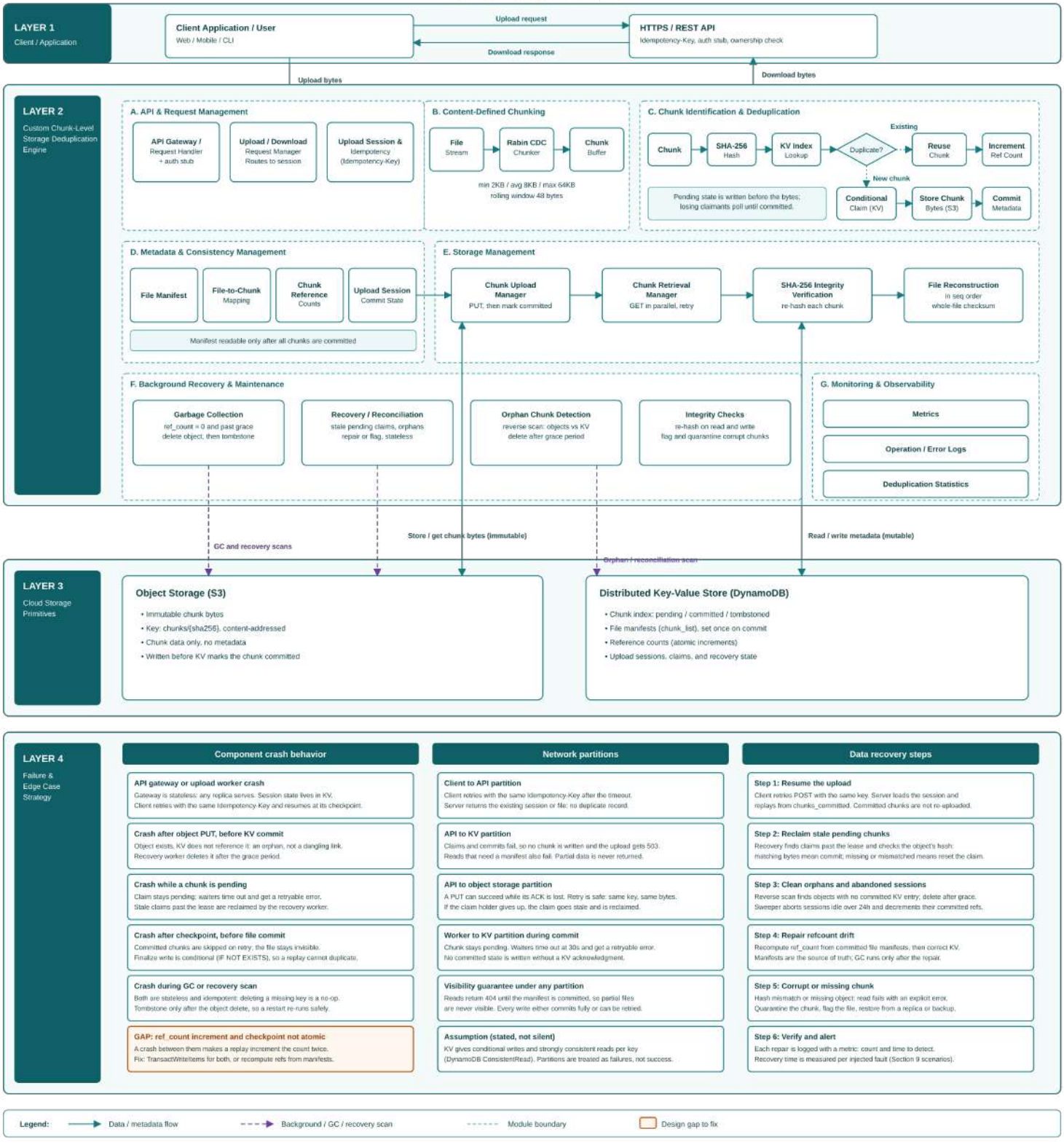
Chunk-Level Storage Deduplication Engine

SYSTEM ARCHITECTURE



LAYER 4 — Failure & Edge Case Strategy

Component crash behavior

API gateway or upload worker crash
Gateway is stateless; any replica serves. Session state lives in KV.
Client retries with the same Idempotency-Key and resumes at its checkpoint.

Crash after object PUT, before KV commit
Object exists, KV does not reference it: an orphan, not a dangling link.
Recovery worker deletes it after the grace period.

Crash while a chunk is pending
Claim stays pending; waiters time out and get a retryable error.
Stale claims past the lease are reclaimed by the recovery worker.

Crash after checkpoint, before file commit
Committed chunks are skipped on retry; the file stays invisible.
Finalize write is conditional (IF NOT EXISTS), so a replay cannot duplicate.

Crash during GC or recovery scan
Both are stateless and idempotent: deleting a missing key is a no-op.
Tombstone only after the object delete, so a restart re-runs safely.

GAP: ref_count increment and checkpoint not atomic
A crash between them makes a replay increment the count twice.
Fix: TransactWriteItems for both, or recompute refs from manifests.

Network partitions

Client to API partition
Client retries with the same Idempotency-Key after the timeout.
Server returns the existing session or file; no duplicate record.

API to KV partition
Claims and commits fail, so no chunk is written and the upload gets 503.
Reads that need a manifest also fail. Partial data is never returned.

API to object storage partition
A PUT can succeed while its ACK is lost. Retry is safe: same key, same bytes.
If the claim holder gives up, the claim goes stale and is reclaimed.

Worker to KV partition during commit
Chunk stays pending. Waiters time out at 30s and get a retryable error.
No committed state is written without a KV acknowledgment.

Visibility guarantee under any partition
Reads return 404 until the manifest is committed, so partial files
are never visible. Every write either commits fully or can be retried.

Assumption (stated, not silent)
KV gives conditional writes and strongly consistent reads per key
(DynamoDB ConsistentRead). Partitions are treated as failures, not success.

Data recovery steps

Step 1: Resume the upload
Client retries POST with the same key. Server loads the session and
replays from chunks_committed. Committed chunks are not re-uploaded.

Step 2: Reclaim stale pending chunks
Recovery finds claims past the lease and checks the object's hash:
matching bytes mean commit; missing or mismatched means reset the claim.

Step 3: Clean orphans and abandoned sessions
Reverse scan finds objects with no committed KV entry; delete after grace.
Sweeper aborts sessions idle over 24h and decrements their committed refs.

Step 4: Repair refcount drift
Recompute ref_count from committed file manifests, then correct KV.
Manifests are the source of truth; GC runs only after the repair.

Step 5: Corrupt or missing chunk
Hash mismatch or missing object: read fails with an explicit error.
Quarantine the chunk, flag the file, restore from a replica or backup.

Step 6: Verify and alert
Each repair is logged with a metric: count and time to detect.
Recovery time is measured per injected fault (Section 9 scenarios).

Legend: Data / metadata flow · Background / GC / recovery scan · Module boundary · Design gap to fix